Task 3 Project Report

RTL Verification, SystemVerilog, Memory Design, and a Parameterized ALU

Project overview

This project develops a configurable arithmetic logic unit (ALU), small RAM and ROM blocks, selected SystemVerilog RTL examples, and a directed verification environment. It organizes design, assertion, test planning, and documentation artifacts for review.

Implemented design scope

The ALU supports addition, subtraction, multiplication, AND, OR, XOR, NOT, logical left/right shifts, and equality comparison. The default width is 8 bits; the parameterized testbench instantiates 8-, 16-, and 32-bit configurations. The 16 x 8 single-port RAM uses synchronous writes and asynchronous reads. The 16 x 8 ROM contains a fixed documented lookup table.

Selector	Function	Defined status
0-2	ADD, SUB, MUL	Carry and overflow semantics documented
3-6	AND, OR, XOR, NOT	Bitwise operations
7-8	SHL, SHR	Logical shifts; amount from low selector-width bits of B
9	EQ	One-bit equality result, zero extended
A-F	Reserved	Deterministic zero result

SystemVerilog examples use logic, always_comb, always_ff, a packed enum FSM state, and parameterized vectors. Immediate assertion monitors check legal state encodings, counter bounds, reset behavior, and unknown ALU outputs. Concurrent SVA syntax is explained separately because support depends on the simulator.

Verification approach

The test plan combines directed and repeatable pseudo-random operation checks, boundary conditions, invalid ALU selector behavior, width-scalability examples, memory address coverage, write-inhibit behavior, and sequential reset/enable checks. A functional coverage plan defines operation, operand, flag, width, address, and reset bins without claiming structural code-coverage percentages. A separate black-box plan defines reset, phase sequence, phase duration, state validity, and output exclusivity scenarios for the Task 2 traffic-light controller.

Test group	Coverage intent	Testbench
ALU directed/random	15 directed plus 64 repeatable pseudo-random checks; flags, shifts, compare, invalid opcode	tb_alu.sv
Parameterized ALU	8-, 16-, and 32-bit instantiations	tb_parameterized_alu.sv
RAM and ROM	Write/read all 16 RAM words, disabled write, compare all 16 ROM values	tb_memory.sv
SystemVerilog blocks	Sync/async reset, enable capture/hold, enum FSM, counter overflow and clock assertions	tb_systemverilog.sv
Task 2 controller	Reset, phase timing, legal states, outputs, edge corner cases	black-box plan in docs/verification_plan.md; not rerun here

Simulation runner

Run `bash scripts/run_all.sh` from the project root. It compiles four testbench tops with Icarus Verilog in SystemVerilog mode and is configured to save transcript logs and VCD traces. GTKWave can inspect the generated traces. The project includes a waveform review guide and identifies the signals and boundary events to inspect.

Evidence and bug status

All four testbench tops were run in Ubuntu/WSL with Icarus Verilog. The ALU bench passed 15 directed and 64 repeatable pseudo-random checks; the parameterized ALU passed at 8, 16, and 32 bits; the memory bench passed all 16 RAM write/read locations, write inhibit, and all 16 ROM words; the SystemVerilog bench passed reset, flip-flop capture, register enable/hold, enum FSM, counter overflow, and clock synchronization checks. Logs and VCD traces are saved under simulation/. Three genuine GTKWave screenshots are included for ALU, RAM/ROM, and reset/FSM/counter traces. Concise observations are in docs/waveform_analysis.md. No confirmed design bug has been observed.

Repository and deliverable map

Deliverable	Location
Verilog/SystemVerilog RTL	rtl/
Verification testbenches	testbench/
Assertion source	assertions/design_assertions.sv
Simulation logs / VCD	simulation/results/ and simulation/vcd/ (generated after passing run)
GTKWave screenshots	waveforms/alu_waveform.png, memory_ram_rom.png, reset_fsm_counter.png
ALU and memory block diagrams	docs/diagrams/
Verification plan and test case matrix	docs/verification_plan.md and docs/test_case_matrix.md
Functional coverage plan	docs/coverage_plan.md
Bug report	docs/bug_report.md
README and project report	README.md and docs/task3_project_report.pdf
Daily progress log	progress_log/progress.md

Reproduction steps

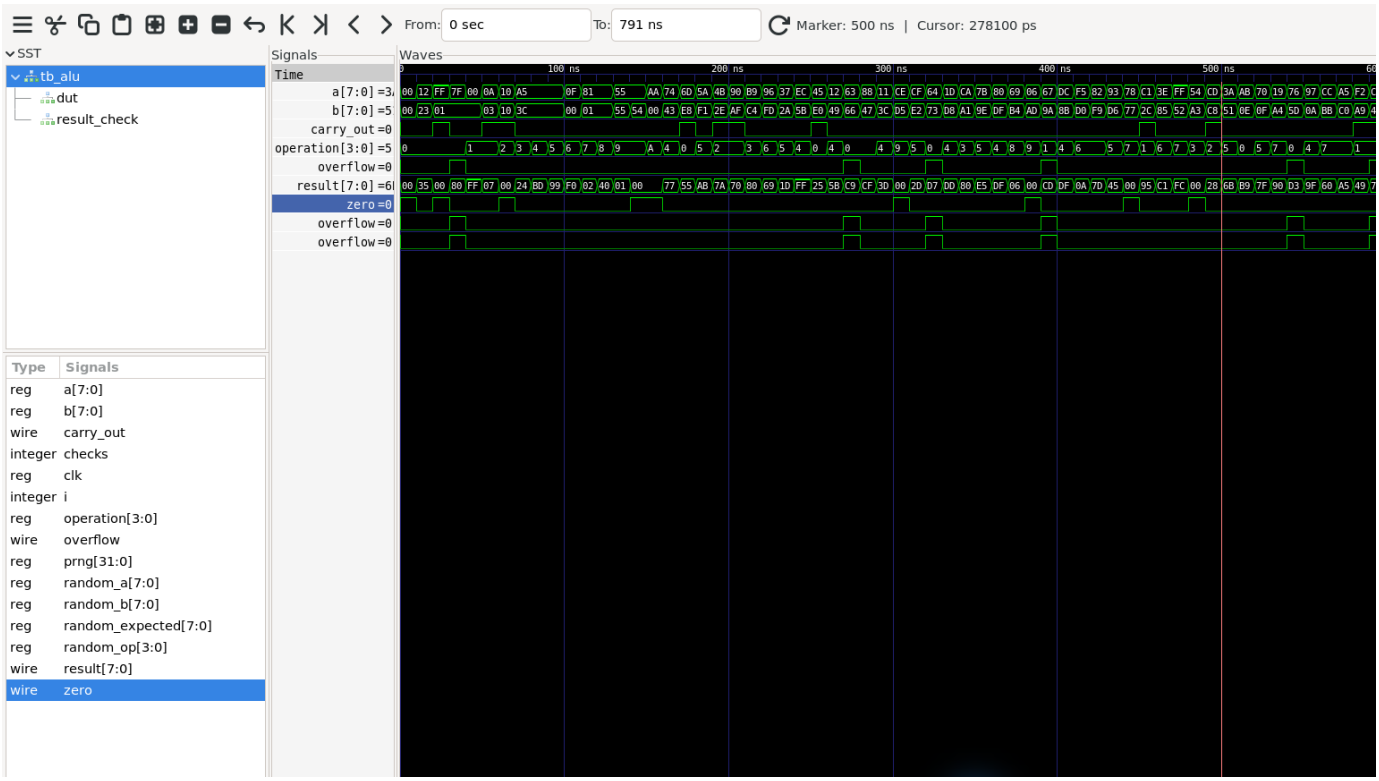
1. Install or activate Icarus Verilog with SystemVerilog support and VVP. 2. From the repository root, run `bash scripts/run_all.sh`. 3. Review the logs under `simulation/results/`. 4. Open each VCD in GTKWave and compare it with the representative captures and notes in `waveforms/` and `docs/waveform_analysis.md`.

Status note

The saved simulation logs report all four testbench tops passing. Three representative GTKWave views are included. Review the simulator transcripts for detailed checks and consult docs/waveform_analysis.md for the visible signals and scope limits.

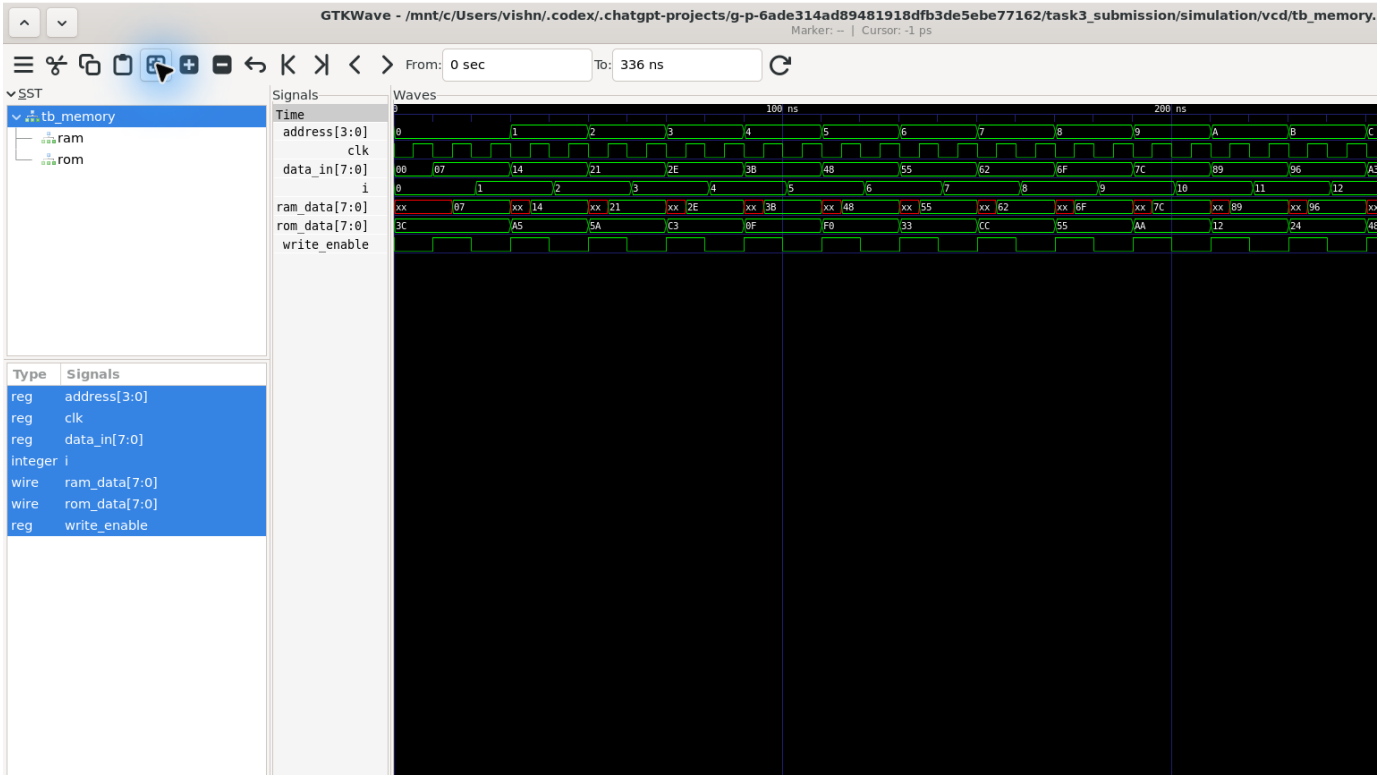
GTKWave evidence

ALU inputs, operation selector, result, and flags across directed and repeatable pseudo-random vectors.



GTKWave evidence: RAM and ROM

Address, clock, write control, input data, and memory outputs across the simulated address sequence.



GTKWave evidence: reset, FSM, and counter

Reset and clock, counter progression, FSM state/next state, and traffic-light outputs.

